Aqua Protocol: StableSwap Mathematical Model

Aqua Protocol

The Aqua Protocol is a Soroban-based Automated Market Maker (AMM). Source code repository: GitHub - soroban-amm

1 Equation - StableSwap Mathematics

The Aquarius protocol implements a StableSwap algorithm similar to Curve Finance for its stable liquidity pool. This document details the mathematical models that govern the behavior of the Smart Contract.

1.1 The StableSwap Invariant

Unlike Uniswap's constant product model ($x \cdot y = k$), StableSwap uses a more complex invariant function:

$$A \cdot n^n \cdot \sum_{i=1}^{n} x_i + D = A \cdot D \cdot n^n + \frac{D^{n+1}}{n^n \cdot \prod_{i=1}^{n} x_i} \tag{1}$$

where:

- x_i are the normalized balances of each token
- n is the number of tokens in the pool
- A is the amplification coefficient
- D is the invariant

Behavior of the invariant:

- When $A \to 0$: behaves like constant sum AMM ($\sum x_i = k$)
- When $A \to \infty$: behaves like constant product AMM ($\prod x_i = k$)
- Intermediate A: optimal balance for stable tokens

2 Core Mathematical Functions

2.1 Swap: Token Exchange

Function signature: `fn swap(user, in_idx, out_idx, in_amount, out_min) → out_amount`

1. Normalization:
$$xp_i = \text{reserves}_i \cdot \text{precisionmul}_i \tag{2}$$
2. Input update:
$$x' = xp_{\text{in\_idx}} + \text{in\_amount} \cdot \text{precision\_mul}_{\text{in\_idx}} \tag{3}$$
3. New output balance:
$$y = \text{\_get\_y}(\text{in\_idx}, \text{out\_idx}, x', xp) \tag{4}$$
4. Raw output:
$$dy_{\text{raw}} = xp_{\text{out\_idx}} - y - 1 \tag{5}$$
5. Fee application:
$$dy_{\text{fee}} = dy_{\text{raw}} \cdot \frac{\text{fee}}{\text{FEE\_DENOMINATOR}} \tag{6}$$
6. Final output:
$$\text{out\_amount} = \frac{dy_{\text{raw}} - dy_{\text{fee}}}{\text{precision\_mul}_{\text{out\_idx}}} \tag{7}$$
7. Minimum condition: *If* out_amount $<$ out_min, *transaction fails.*

2.2 get_y: Output Balance Calculation

Function signature: `fn get_y(in_idx, out_idx, x, xp) → y`

1. Compute invariant:
$$D = \text{\_get\_d}(xp, A)$$
2. Coefficients:
$$c = \frac{D^{n+1}}{(A \cdot n^n) \cdot \prod_{i \neq \text{out\_idx}} x_i}, \quad s = \sum_{i \neq \text{out\_idx}} x_i, \quad b = s + \frac{D}{A \cdot n}$$
3. Newton iteration:
$$y_{i+1} = \frac{y_i^2 + c}{2y_i + b - D}$$
4. Stop when $|y_{i+1} - y_i| \leq 1$

2.3 Amplification Coefficient A

Function signature: `fn a() → amp`

A evolves over time via:

$$A(t) = \begin{cases} A_0 + (A_1 - A_0) \cdot \frac{t-t_0}{t_1-t_0}, & A_1 > A_0,\ t_0 \le t < t_1 \\ A_0 - (A_0 - A_1) \cdot \frac{t-t_0}{t_1-t_0}, & A_1 < A_0,\ t_0 \le t < t_1 \\ A_1, & t \ge t_1 \end{cases}$$

2.4 Invariant D Calculation

Function signature: `fn _get_d(xp, amp) → D`

1. Sum:
$$S = \sum_{i=1}^{n} x_i$$
2. Initial $D = S$ if $S \neq 0$
3. Iterative steps:
$$D_{p,j} = D_j \cdot \prod_{i=1}^{n} \frac{D_j}{n \cdot x_i}$$
$$D_{j+1} = \frac{(ann \cdot S + D_{p,j} \cdot n) \cdot D_j}{(ann - 1) \cdot D_j + (n+1) \cdot D_{p,j}}, \quad \text{with } ann = A \cdot n^n$$
4. Convergence when $|D_{j+1} - D_j| \le 1$

3 Currency Implications

- **Reduced slippage** for stable token swaps.
- **Adaptive flexibility** via A.
- **Stable price zone** near token parity.
- **Deviation resistance** discouraging arbitrage.

Annexes

- function a()
- function swap()